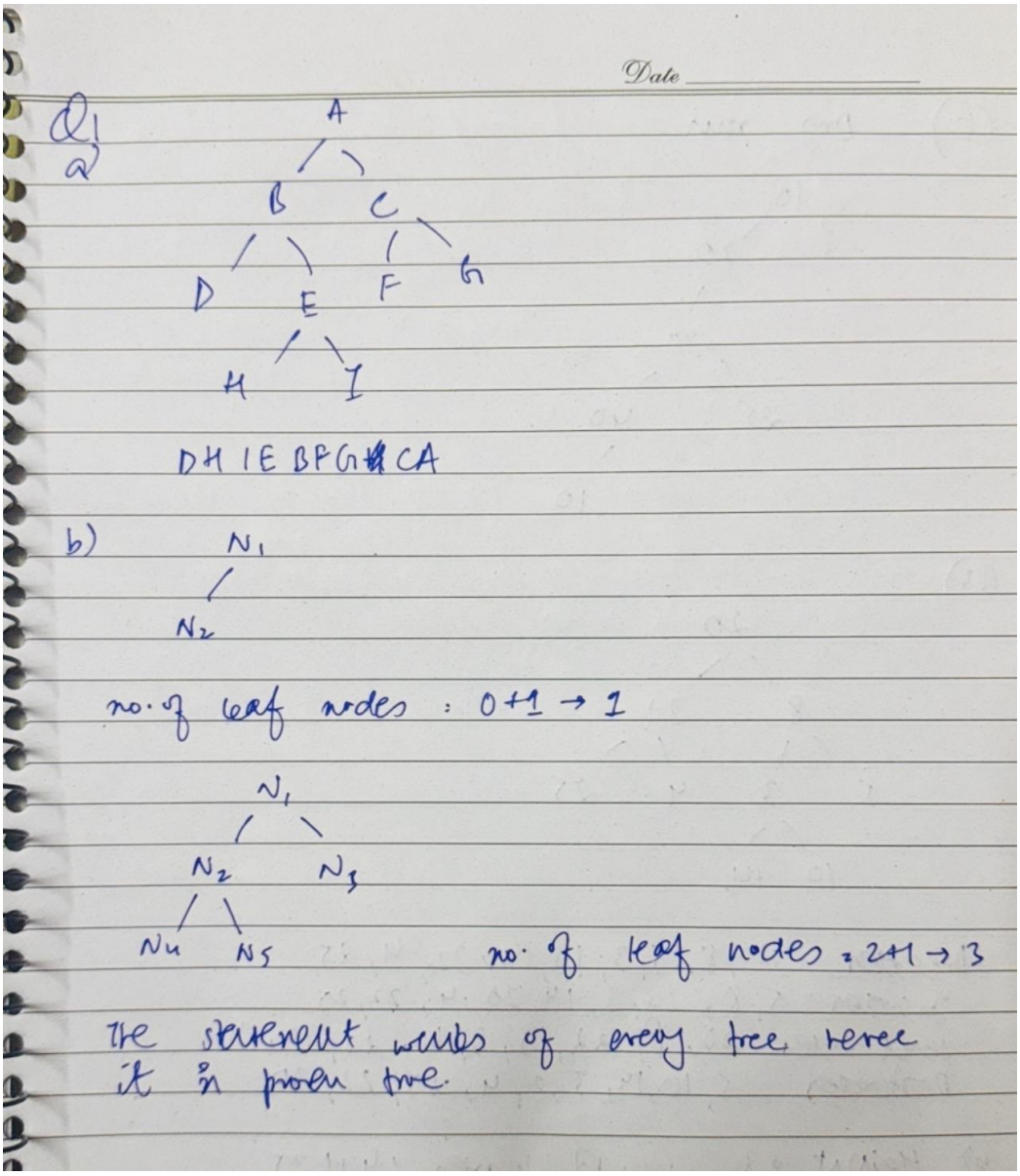
Raja Muhammad Taha

25K-0086

Q1)



Q2)

```
#include <iostream>
using namespace std;

class Node
{
public:
int data;
   Node* left;
   Node* right;

   Node(int value)
   {
      data = value;
left = NULL;
right = NULL;
   }
};

class BST
{
   Node* root;

   Node* insert(Node* node, int key)
```

```
  {
      if(node == NULL)
return new Node(key);
if(key < node->data)
node->left = insert(node-
>left, key);       else if(key >
node->data)          node-
>right = insert(node->right,
key);

      return node;
  }

  bool search(Node* node, int key)
  {
      if(node == NULL)
return false;

      if(node->data == key)
return true;

      if(key < node->data)
return search(node->left, key);

      return search(node->right, key);
  }
```

```
   Node* findMinNode(Node* node)
   {
      while(node != NULL && node->left != NULL)
node = node->left;

      return node;
   }

   Node* findMaxNode(Node* node)
   {
      while(node != NULL && node->right != NULL)
node = node->right;

      return node;
   }

   Node* deleteNode(Node* node, int key)
   {
      if(node == NULL)
return NULL;

      if(key < node->data)          node->left =
deleteNode(node->left, key);
```

```
    else if(key > node->data)          node->right =
deleteNode(node->right, key);

else
    {
      if(node->left == NULL && node->right == NULL)          {
        delete node;
return NULL;
      }

      else if(node->left == NULL)
      {
        Node* temp = node->right;
delete node;            return temp;
      }

      else if(node->right == NULL)
      {
        Node* temp = node->left;
delete node;            return
temp;
      }
else
      {
```

```
            Node* temp = findMinNode(node->right);
node->data = temp->data;            node->right =
deleteNode(node->right, temp->data);
          }
       }

       return node;
   }

   int height(Node* node)
   {
       if(node == NULL)
return -1;

       int leftHeight = height(node->left);
int rightHeight = height(node->right);

       if(leftHeight > rightHeight)
return leftHeight + 1;

       return rightHeight + 1;
   }

   void inorder(Node* node)
   {
       if(node == NULL)
```

```
        return;

    inorder(node->left);
cout << node->data << " ";
inorder(node->right);
  }

public:

  BST()
  {
    root = NULL;
  }

  void insert(int key)
  {
    root = insert(root, key);
  }

  bool search(int key)
  {
    return search(root, key);
  }

  void deleteKey(int key)
```

```
    {
        root = deleteNode(root, key);
    }

    int findMin()
    {
        Node* temp = findMinNode(root);

        if(temp == NULL)
return -1;

        return temp->data;
    }

    int findMax()
    {
        Node* temp = findMaxNode(root);

        if(temp == NULL)
return -1;

        return temp->data;
    }

    int height()
    {
```

```
    return height(root);
  }

  int inorderSuccessor(int key)
  {
    Node* current = root;
    Node* successor = NULL;

    while(current != NULL)
    {
      if(key < current->data)
      {
        successor = current;
current = current->left;
      }
      else if(key > current->data)
      {
        current = current->right;
      }
else
      {
        if(current->right != NULL)
        {
          successor = findMinNode(current->right);
        }
```

```
break;
          }
       }

       if(current == NULL)
return -1;        if(successor ==
NULL)           return -1;

       return successor->data;
   }

   void display()
   {
      inorder(root);
cout << endl;
   }
};

int main()
{
   BST tree;

   tree.insert(10);
tree.insert(20);     tree.insert(30);
```

```
tree.insert(40);     tree.insert(50);
tree.insert(25);

   cout << "Inorder: ";     tree.display();

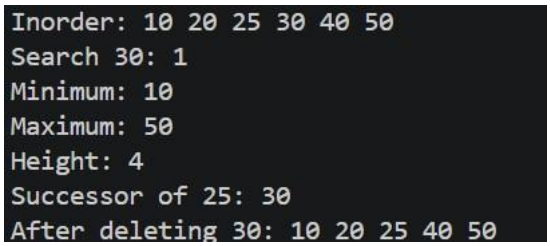
   cout << "Search 30: " << tree.search(30) << endl;     cout <<
"Minimum: " << tree.findMin() << endl;     cout << "Maximum: " <<
tree.findMax() << endl;     cout << "Height: " << tree.height() <<
endl;     cout << "Successor of 25: " << tree.inorderSuccessor(25) <<
endl;

   tree.deleteKey(30);

   cout << "After deleting 30: ";
tree.display();

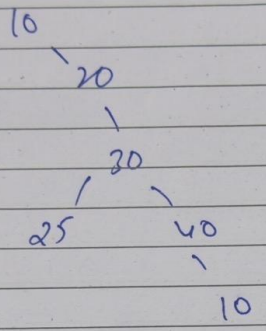
   return 0;
}
```

Output:

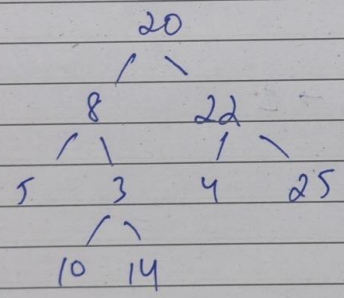


Date

Q2) Dry run



Q3)

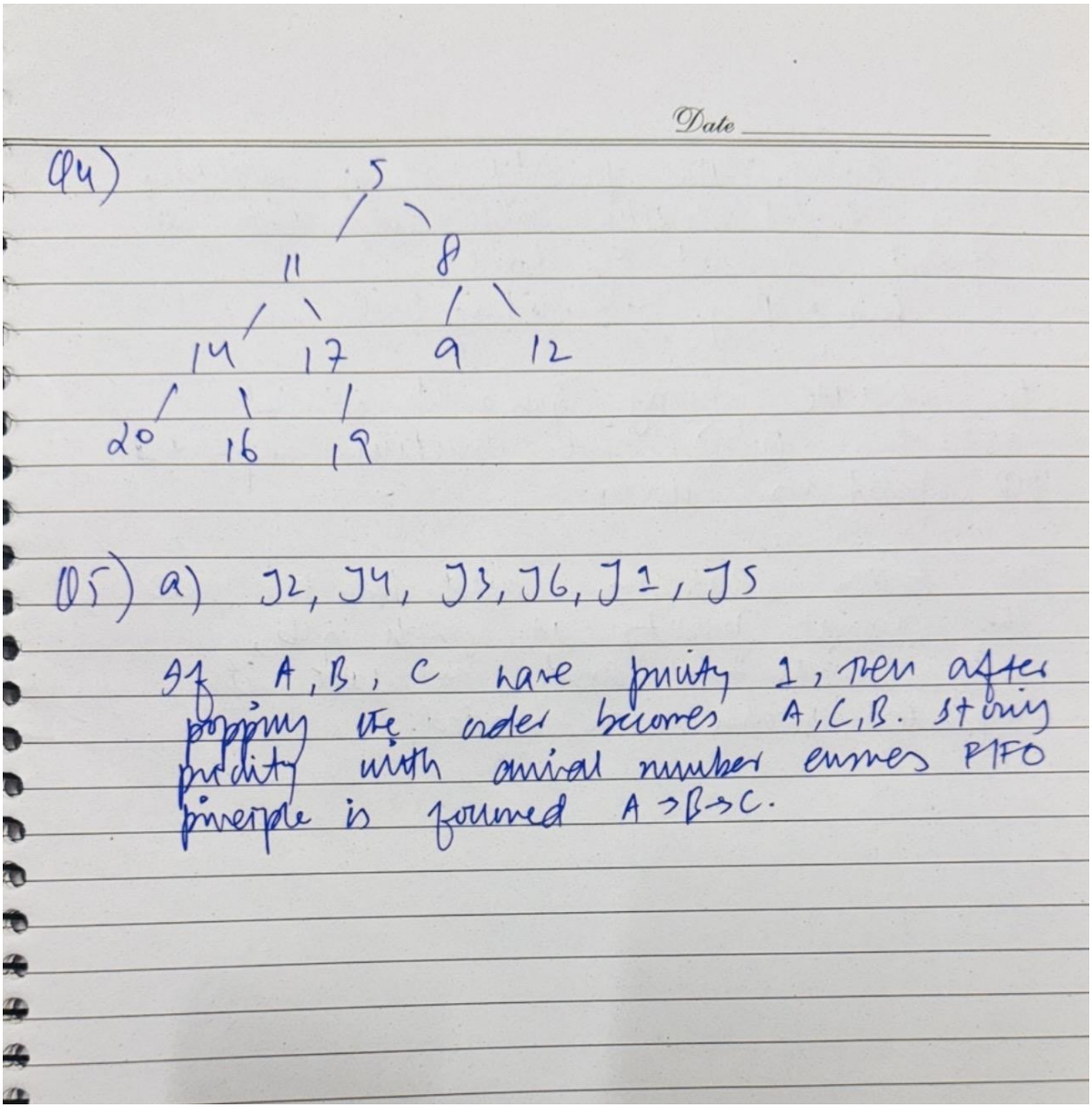


Preorder 20, 8, 5, 3, 10, 14, 22, 4, 25
Inorder 5, 8, 10, 3, 14, 20, 4, 22, 25
Level-order 20, 8, 22, 5, 3, 4, 25, 10, 14
Postorder 5, 10, 14, 3, 8, 4, 25, 22, 30

b) Height = 3, no. of leaves = 4+1 = 5

Q4)

Diagram:



Code:

```
#include <iostream> using
namespace std; class
MinHeap
{
   int heap[100];
int size;

public:

   MinHeap()
   {       size
= 0;
   }

   void insert(int x)
   {
      heap[size] = x;
int i = size;
size++;

      while(i > 0 && heap[i] < heap[(i-1)/2])
      {
         int temp = heap[i];
heap[i] = heap[(i-1)/2];
heap[(i-1)/2] = temp;
```

```
          i = (i-
1)/2;        }
   }

   int extractMin()
   {        if(size
== 0)
return -1;

       int min = heap[0];

       heap[0] = heap[size-1];
       size--;
        int i =
0;

       while(2*i+1 < size)
       {
          int smallest = 2*i+1;

          if(smallest+1 < size && heap[smallest+1] < heap[smallest])
             smallest++;

          if(heap[i] < heap[smallest] || heap[i] == heap[smallest])
             break;
```

```
            int temp = heap[i];
            heap[i] = heap[smallest];
heap[smallest] = temp;

            i = smallest;
        }

        return min;
    }

    void decreaseKey(int index, int value)
    {
        if(index < 0 || index >= size || value > heap[index])
            return;

        heap[index] = value;

        while(index > 0 && heap[index] < heap[(index-1)/2])
        {
            int temp = heap[index];
heap[index] = heap[(index-1)/2];
heap[(index-1)/2] = temp;

            index = (index-1)/2;
        }
```

```
    }

    void display()
    {
        for(int i=0; i<size; i++)
cout << heap[i] << " ";

        cout << endl;
    }
};

int main()
{
    MinHeap h;

    int a[] = {5,11,8,14,17,9,12,20,16,19};

    for(int i=0; i<10; i++)
        h.insert(a[i]);

    h.insert(6);
    h.insert(4);

    h.display();
```

```
    cout << "Extracted: " << h.extractMin() << endl;
   cout << "Extracted: " << h.extractMin() << endl;

   h.decreaseKey(7,3);

   h.display();

   return 0;
```

```
4 6 5 14 11 8 12 20 16 19 17 9
Extracted: 4
Extracted: 5
3 6 8 11 17 9 12 14 16 19
```

Q5)

a) and b)

c)

```
#include <iostream> using
namespace std;

class Job
{
public:
   char name[10];
   int priority;
int arrival;
```

```
};

class PriorityQueue
{
   Job heap[100];
   int size;
int counter;

public:

   PriorityQueue()
   {       size = 0;
counter = 0;
   }

   void push(const char name[], int priority)
   {       int i =
size;
       int j =
0;

      while(name[j] != '\0' && j < 9)
      {
         heap[i].name[j] = name[j];
         j++;
      }
```

```
        heap[i].name[j] = '\0';

        heap[i].priority = priority;
heap[i].arrival = ++counter;

        size++;

        while(i > 0)
        {
            int parent = (i-1)/2;

            if(heap[i].priority > heap[parent].priority)
                break;

            if(heap[i].priority == heap[parent].priority &&
heap[i].arrival > heap[parent].arrival)
                break;

            Job temp = heap[i];
heap[i] = heap[parent];
heap[parent] = temp;

            i = parent;
        }
    }
```

```
   void pop()
   {       if(size
== 0)
       {
          cout << "Empty Queue" << endl;
          return;
       }

       cout << heap[0].name << endl;

       heap[0] = heap[size-1];
       size--;
        int i =
0;

       while(2*i+1 < size)
       {
          int smallest = 2*i+1;

          if(smallest+1 < size)
          {
             if(heap[smallest+1].priority          <          heap[smallest].priority          ||
(heap[smallest+1].priority == heap[smallest].priority &&          heap[smallest+1].arrival
< heap[smallest].arrival))          smallest++;
          }
```

```
        if(heap[i].priority < heap[smallest].priority ||
(heap[i].priority == heap[smallest].priority &&
heap[i].arrival < heap[smallest].arrival))
            break;

        Job temp = heap[i];
heap[i] = heap[smallest];
heap[smallest] = temp;

        i = smallest;
      }
   }

   void peek()
   {       if(size == 0)           cout <<
"Empty Queue" << endl;
      else
        cout << heap[0].name << endl;
   }
};
int main()
{
   PriorityQueue q;

   q.push("J1",3);
```

```
    q.push("J2",1);
    q.push("J3",2);

    q.pop();

    q.push("J4",1);
    q.push("J5",3);

    q.pop();
    q.pop();

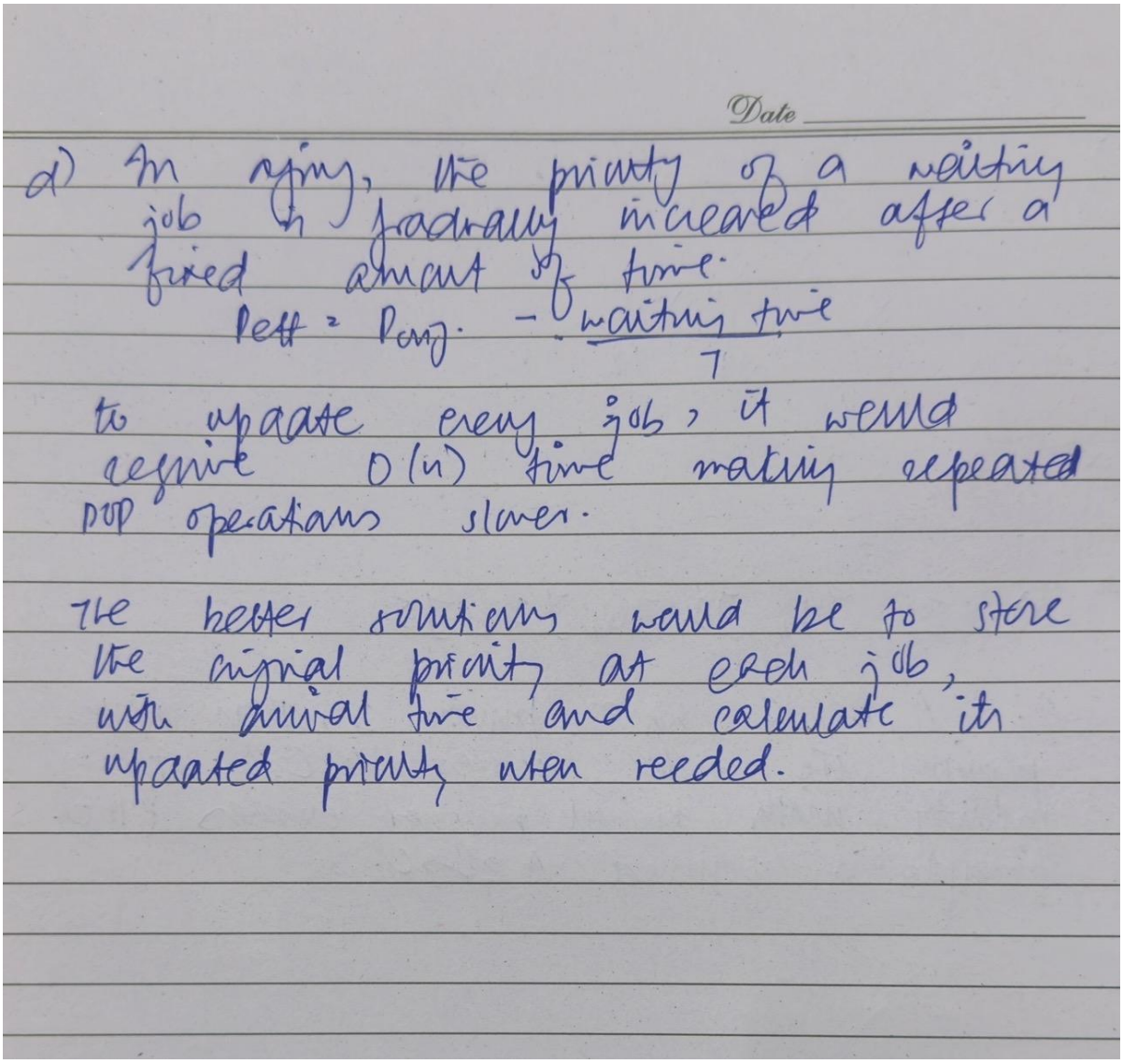
    q.push("J6",2);

    q.pop();
    q.pop();
    q.pop();

    return 0;
}
```

```
J2
J4
J3
J6
J1
J5
```

d)



Q6)

```
#include <iostream> using

namespace std;


class Student

{

public:
```

```
   int id;
   char name[40];
char department[30];
double cgpa;
};
void merge(Student a[], Student temp[], int l, int m, int r)
{    int i = l;
int j = m+1;
int k = l;

   while(i <= m && j <= r)
   {
      if(a[i].cgpa >= a[j].cgpa)
temp[k++] = a[i++];
       else
temp[k++] = a[j++];
   }

   while(i <= m)
temp[k++] = a[i++];

   while(j <= r)
temp[k++] = a[j++];

   for(int x=l; x<=r; x++)
a[x] = temp[x];
```

```
}

void mergeSort(Student a[], Student temp[], int l, int r)
{    if(l >= r)
return;

   int m = (l+r)/2;

   mergeSort(a,temp,l,m);
mergeSort(a,temp,m+1,r);

   merge(a,temp,l,m,r);
}

int main()
{    const int n =
5;

   Student a[n], temp[n];

   double rector, dean;

   cout << "Enter Rector threshold: ";
cin >> rector;
```

```
   cout << "Enter Dean threshold: ";
cin >> dean;

   for(int i=0; i<n; i++)
   {
      cout << "Enter ID, Name, Department, CGPA: ";

      cin >> a[i].id;        cin
>> a[i].name;        cin >>
a[i].department;        cin >>
a[i].cgpa;
   }

   mergeSort(a,temp,0,n-1);

   cout << "\nFINAL MERIT LIST\n";

   for(int i=0; i<n; i++)
   {
      cout << a[i].id << " ";        cout
<< a[i].name << " ";        cout <<
a[i].department << " ";        cout <<
a[i].cgpa << " ";

      if(a[i].cgpa >= rector)
cout << "Rector's List";
```

```
        else if(a[i].cgpa >= dean)
            cout << "Dean's List";
        else            cout <<
"No Award";

        cout << endl;
    }

    return 0;
}
```

```
Enter Rector threshold: 3.75
Enter Dean threshold: 3.5
Enter ID, Name, Department, CGPA: 0077 Ayan AI 2.9
Enter ID, Name, Department, CGPA: 0086 Raja AI 3.8
Enter ID, Name, Department, CGPA: 0098 Hasan AI 2.3
Enter ID, Name, Department, CGPA: 0166 Murtaza AI 2.5
Enter ID, Name, Department, CGPA: 0070 Iliyaan AI 3.4

FINAL MERIT LIST
86 Raja AI 3.8 Rector's List
70 Iliyaan AI 3.4 No Award
77 Ayan AI 2.9 No Award
166 Murtaza AI 2.5 No Award
98 Hasan AI 2.3 No Award
```

Q7)

```
#include <iostream> using
namespace std;

class Parcel
{
public:
```

```
    int tracking;
   char sender[40];
char city[40]; int
region;    int
originalOrder;
};

void swapParcel(Parcel &a, Parcel &b)
{
    Parcel temp = a;
a = b;     b = temp;
}

int getDigit(int number, int exp)
{
    return (number/exp)%10;
}

void radixSort(Parcel a[], int low, int high, int exp)
{
    if(high-low <= 1 || exp == 0)
        return;

    int count[10] = {0};

   for(int i=low; i<high; i++)     count[getDigit(a[i].tracking,exp)]++;
```

```
    int start[10], next[10], end[10];

    int pos = low;

    for(int i=0; i<10; i++)
    {
        start[i] = pos;
next[i] = pos;

        pos += count[i];

        end[i] = pos;
    }

    for(int i=0; i<10; i++)
    {
        while(next[i] < end[i])
        {
            int d = getDigit(a[next[i]].tracking,exp);
             if(d == i)
next[i]++;

           else
            {
                swapParcel(a[next[i]],a[next[d]]);
     next[d]++;
```

```
        }
      }
   }

   for(int i=0; i<10; i++)
   {
      if(count[i] > 1)
radixSort(a,start[i],end[i],exp/10);
   }
}

bool greaterParcel(Parcel a, Parcel b)
{
   if(a.region != b.region)
return a.region > b.region;

   return a.originalOrder > b.originalOrder;
}

void heapify(Parcel a[], int n, int i)
{   int largest =
 i; int left =
 2*i+1; int right
 = 2*i+2;
```

```
    if(left < n && greaterParcel(a[left],a[largest]))
largest = left;

    if(right < n && greaterParcel(a[right],a[largest]))
largest = right;

    if(largest != i)
    {
        swapParcel(a[i],a[largest]);

        heapify(a,n,largest);
    }
}

void regionSort(Parcel a[], int n)
{    for(int i=n/2-1; i>=0; i-
-)        heapify(a,n,i);

    for(int i=n-1; i>0; i--)
    {
       swapParcel(a[0],a[i]);
        heapify(a,i,0);
    }
}

int main()
```

```
{    int
n;

   cout << "Enter number of parcels: ";
cin >> n;

   Parcel* a = new Parcel[n];

   for(int i=0; i<n; i++)
   {
      cout << "Enter tracking, sender, city and region: ";

      cin >> a[i].tracking;
cin >> a[i].sender;
cin >> a[i].city;       cin
>> a[i].region;

      a[i].originalOrder = i;
   }

  radixSort(a,0,n,100000000);
   cout << "\nSorted by Tracking Number:\n";

   for(int i=0; i<n; i++)       cout
<< a[i].tracking << " ";
```

```
    regionSort(a,n);

    cout << "\n\nSorted by Region Code:\n";

    for(int i=0; i<n; i++)
    {
        cout << a[i].region << " ";
cout << a[i].sender << " ";
cout << a[i].city << endl;
    }

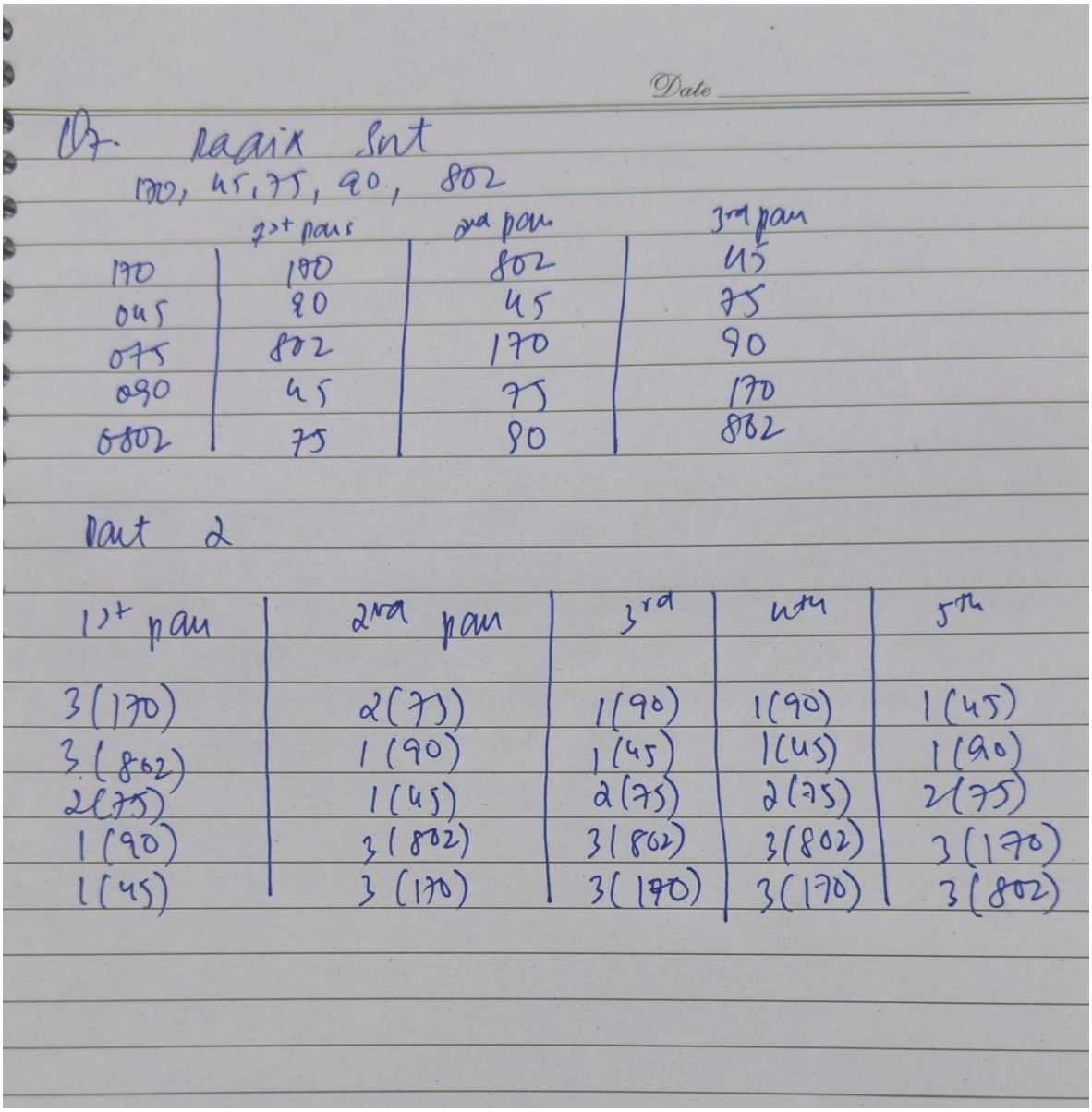
    delete[] a;

    return 0;
}
```

```
Enter number of parcels: 5
Enter tracking, sender, city and region: 170 Ali Karachi 3
Enter tracking, sender, city and region: 45 Sara Islamabad 1
Enter tracking, sender, city and region: 75 Bilal Multan 2
Enter tracking, sender, city and region: 90 Hira Lahore 1
Enter tracking, sender, city and region: 802 Omar Peshawar 3

Sorted by Tracking Number:
45 75 90 170 802

Sorted by Region Code:
1 Sara Islamabad
1 Hira Lahore
2 Bilal Multan
3 Ali Karachi
3 Omar Peshawar
```



Q8)

```
#include <iostream> using

namespace std; const int

N = 4;


int maze[N][N] =

{
```

```
    {1,1,0,1},
    {1,1,1,1},
    {0,1,1,0},
    {1,1,1,1}
};

bool visited[N][N];

int pathR[100]; int
pathC[100];

int total = 0;

void solve(int r, int c, int len)
{
    if(r<0 || c<0 || r>=N || c>=N)
        return;

    if(maze[r][c] == 0 || visited[r][c])
return;

    visited[r][c] = true;

    pathR[len] = r;
pathC[len] = c;
```

```
    if(r==N-1 && c==N-1)
    {
        total++;

        cout << "Path " << total << ": ";

        for(int i=0; i<=len; i++)            cout << "(" <<
pathR[i] << "," << pathC[i] << ") ";

        cout << endl;
    }

    else
    {
        solve(r-1,c,len+1);

        solve(r+1,c,len+1);

        solve(r,c-1,len+1);

        solve(r,c+1,len+1);
    }
    visited[r][c] = false;
}

int main()
```

```
{
solve(0,0,0);

   if(total == 0)        cout <<
"No path exists";

   else        cout << "Total paths: "
<< total;

   return 0;
}
```

```
Path 1: (0,0) (1,0) (1,1) (2,1) (3,1) (3,2) (3,3)
Path 2: (0,0) (1,0) (1,1) (2,1) (2,2) (3,2) (3,3)
Path 3: (0,0) (1,0) (1,1) (1,2) (2,2) (3,2) (3,3)
Path 4: (0,0) (1,0) (1,1) (1,2) (2,2) (2,1) (3,1) (3,2) (3,3)
Path 5: (0,0) (0,1) (1,1) (2,1) (3,1) (3,2) (3,3)
Path 6: (0,0) (0,1) (1,1) (2,1) (2,2) (3,2) (3,3)
Path 7: (0,0) (0,1) (1,1) (1,2) (2,2) (3,2) (3,3)
Path 8: (0,0) (0,1) (1,1) (1,2) (2,2) (2,1) (3,1) (3,2) (3,3)
Total paths: 8
```

Q9)

```
#include <iostream>
#include <cstdlib> using
namespace std; const
int R = 6; const int C =
8;

int seatRow[4]; int
seatCol[4];
```

```
bool usedRow[R]; bool
usedCol[C];

int total = 0;

bool safe(int r, int c, int k)
{
   if(usedRow[r] || usedCol[c])
return false;

   if(r == 3 && c == 3)
return false;

   for(int i=0; i<k; i++)
   {
      if(abs(r-seatRow[i]) == abs(c-seatCol[i]))
return false;
   }

   return true;
}

void arrange(int k)
{    if(k ==
4)
   {
```

```
        total++;

        cout << "Arrangement " << total << ": ";

        for(int i=0; i<4; i++)
        {
           cout << "(" << seatRow[i] << ",";
cout << seatCol[i] << ") ";
        }

        cout << endl;

        return;
    }

    int startRow = 1;
int endRow = R;

    if(k == 0)
    {
        startRow = 0;
endRow = 1;
    }

    for(int r=startRow; r<endRow; r++)
    {
```

```
      for(int c=0; c<C; c++)
      {
if(safe(r,c,k))
         {
            seatRow[k] = r;
seatCol[k] = c;

            usedRow[r] = true;
usedCol[c] = true;

            arrange(k+1);

            usedRow[r] = false;
usedCol[c] = false;
         }
      }
   }
}

int main()
{
arrange(0);

   cout << "\nTotal valid arrangements: " << total << endl;

   return 0;
```

}

```
Arrangement 24334: (0,7) (5,6) (4,4) (3,0)
Arrangement 24335: (0,7) (5,6) (4,4) (3,1)
Arrangement 24336: (0,7) (5,6) (4,4) (3,2)

Total valid arrangements: 24336
```

Q10) #include

<iostream> using

namespace std;

int digits[20]; int

chosen[20];

int M,N,S; int

total = 0;

void generate(int start, int depth, int sum)

{

if(depth == N)

{

if(sum == S)

{

total++;

for(int i=0; i<N; i++)

cout << chosen[i];

cout << endl;

```
        }


        return;
    }


    for(int i=start; i<M; i++)
    {
        if(sum + digits[i] > S)
continue;


        chosen[depth] = digits[i];


        generate(i+1,depth+1,sum+digits[i]);
    }
}


int main()
{
    cout << "Enter number of allowed digits: ";
cin >> M;
    cout << "Enter digits in increasing order: ";


    for(int i=0; i<M; i++)
cin >> digits[i];
```

```
    cout << "Enter N and S: ";
cin >> N >> S;

    generate(0,0,0);

    if(total == 0)        cout << "No valid
combination exists";

    else        cout << "Total combinations: "
<< total;

    return 0;
}
```

```
Enter number of allowed digits: 5
Enter digits in increasing order: 1 2 3 4 5
Enter N and S: 2 6
15
24
Total combinations: 2
```